Проект «Эмулятор микрокалькулятора «Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

Инструкция по файлам M61

Версия инструкции: 31.10.2026

Файл `.m61` - это текстовый сценарий для MK61s mini. Он может просто загрузить программу МК-61 в программную память, а может выполнить подготовку, запустить программу, дождаться ее остановки, проверить стек или регистры и открыть другой сохраненный файл.

В хранилище устройства такие файлы имеют тип `M1`. На USB-диске они видны как обычные файлы с расширением `.M61` или `.m61`.

Кратко

- `.m61` - обычный текстовый файл.
- Одна строка файла - одна команда.
- Разрешенные сценарные команды пишутся в том же виде, что и в терминале устройства; административные команды в сценариях запрещены.
- Пустые строки игнорируются.
- Строки вида `:label` задают метки для переходов.
- `run` запускает загруженную программу МК-61 и продолжает сценарий после ее остановки.
- `run 8` запускает её с адреса 08; аргумент десятичный, без аргумента старт происходит с 00. Диапазон адресов — 0…104 или 0…111 в расширенном режиме.
- `run :label` переходит на метку внутри обычного `.m61`; внутри обработчика `trap` или `bind` это локальный вызов, из которого `ret` возвращает на следующую строку.
- `trap <address> run :label` перехватывает шаг программы до его выполнения.
- `bind <opcode> run :label` заменяет введённую с клавиатуры операцию вызовом обработчика M61; bind key **<**код**>** down/up run :label перехватывает событие кнопки.
- `reinit` возвращает калькулятор в чистое состояние и снимает все активные `trap` и `bind`, после чего сценарий продолжается со следующей строки.
- `ret` возвращает из обработчика, из вложенного `.m61` либо завершает корневой сценарий.
- `print "..."` пишет байты и значения регистров на текущий экран без неявной очистки и без неявного перевода строки.
- `print off` гасит и захватывает экран; `print on` возвращает индикацию МК-61.
- `if ... <command>` выполняет команду только при истинном условии.
- `open <path>` открывает или запускает сохраненный файл и после выхода продолжает сценарий.
- `open? <path>` необязательно открывает файл: если файл или его обработчик недоступен, сценарий молча продолжается.
- loadfont **<**имя**>**... временно выбирает первый доступный FMK: сначала рядом с текущим `.m61`, затем в каталоге `Fonts`; при выходе из M61 исходный шрифт восстанавливается автоматически.
- Квота общая для всех типов и каталогов и зависит от ёмкости SPI NOR: 192 узла на 512 КиБ и 4084 узла на штатной 16-МиБ W25Q128.
- Максимальный размер файла в хранилище - 1536 байта.

Структура файла

Файл состоит из строк:

```
hin 0000 0144004501000049010C0B024A09044C0108070207030806
hin 0024 0E01030000000006003841010307080703030 60E01000006
run
```

В этом примере первые две строки записывают байты в программную память МК-61, а `run` запускает программу.

Имена команд и их числовые аргументы записываются ASCII-символами. Пути и имена файлов записываются в M8. Для имен команд используется нижний регистр, как в примерах: `hin`, `run`, `open`, `if`. Специальная команда записи регистра пишется с большой `R`: `R0= ...`. В M61 пробел после = необязателен: `R0=17` и `R0= 17` равнозначны. Интерактивный терминал сохраняет прежнюю запись с пробелом. Кроме обычного числа правая часть может быть словом `random` или точным 12-тетрадным образом `raw`; эти формы описаны ниже.

Поддерживаются переводы строк LF, CRLF и CR. Управляющие символы внутри строки не нужны. Комментариев в синтаксисе нет: неизвестная команда считается ошибкой и останавливает сценарий.

Одна строка сценария должна помещаться в командный буфер терминала. Практически это означает не более 239 символов полезного текста в строке.

Имена и импорт с USB-диска

При копировании файла на USB-диск устройства расширение определяет тип:

Расширение	Тип в хранилище	Что открывает
`.m61`	`M1`	сценарий МК-61
`.foc`	`F1`	FOCAL
`.tbi`	`B2`	TinyBASIC
`.txt`	`T1`	текстовый просмотрщик
`.state.txt`	`M2`	снимок состояния МК-61
`.fmk`	`f1`	графический шрифт
`.wbmp`	`I1`	графическое изображение
`.ch8`	`C1`	ROM консоли CHIP-8
`.app`	`A1`	приложение APP

Basename ограничен 31 байтом валидного M8. Русские имена сохраняются без транслитерации; USB и `mkc` преобразуют их в Unicode только на внешней границе. Слишком длинное, запрещённое FAT-имя либо регистронезависимый дубликат отклоняются, а не переименовываются молча.

Команды `open` и `run <path>` принимают путь с расширением или без него:

```
open GAME.m61
open GAME
open "Projects/Мои программы/GAME.m61"
open ../lib/COMMON.m61
```

Полностью числовой аргумент `run` — адрес программы, поэтому числовой файл открывайте через `open 8` или с расширением: `run 8.m61`.

Поддерживаются абсолютные и относительные пути, `/` и `\`, компоненты `.` и `..`. Во вложенном сценарии относительный путь считается от папки вызывающего файла. Для одиночного имени сохранён fallback в корень. Если расширение не указано и одинаковый basename имеют файлы разных типов, команда возвращает ошибку ambiguity: нужно дописать расширение.

Загрузка программы МК-61

Главная команда для загрузки программы из `.m61` - `hin`:

```
hin <addr> <hex-bytes>
```

<адрес> - десятичный адрес из четырех цифр. <hex-байты> - слитная строка шестнадцатеричных байтов. Каждый байт занимает две hex-цифры:

```
hin 0000 010203040506
hin 0024 0E01030000000006
```

Адреса `0000`...`0104` относятся к классической памяти на 105 шагов. Если сценарий записывает шаги выше классической области или использует команды регистра RF, устройство автоматически включает расширенную память до 112 шагов.

Когда текущая программа МК-61 сохраняется средствами устройства, она записывается как `.m61` с набором строк `hin`. Автоматический `run` в конец не добавляется. Такой файл при открытии только загружает программу; чтобы он сразу запускался, добавьте строку:

```
run
```

Бинарный образ: load <адрес> <путь.bin>

```
reinit
load 0000 elite.bin
run
```

Адрес — ровно четыре десятичные цифры. Путь отсчитывается от каталога текущего M61-файла; абсолютные пути и кавычки для имён с пробелами тоже поддерживаются. Файл содержит четыре байта CRC32/ISO-HDLC распакованного образа в little-endian, затем один поток ZX0 v2. Полей размера нет.

`load` распаковывает байты подряд с указанного абсолютного адреса. Граница банка в 112 байт пересекается автоматически: один файл может заполнить все 32 банка. Файл `.bin` может занимать до 4096 байт. В AUTO заранее включается банковая память; принудительные 105 шагов сохраняют свой предел.

Загрузка не очищает память и не запускает программу: несколько `load` могут заполнять разные диапазоны. Для новой игры используйте `reinit`. Калькулятор должен быть остановлен. Загрузка из trap-обработчика запрещена.

Неправильный поток, обрыв, байты после маркера конца, выход за адресный диапазон, нехватка банков или несовпадение CRC останавливают M61 до `run`. Частичная запись не откатывается; после ошибки распаковки выполните `reinit` и повторите загрузку либо откройте корневой M61 заново.

Бинарный файл создаёт `tools/programs/program_pack.py`; формат и пример упаковки приведены в руководстве терминала. Команды `ztart`/`zin` удалены. Старые load <номер> и load <путь.m61> сохраняют прежнее значение; для двусмысленного имени с числом и пробелом используйте кавычки вокруг полного пути.

Запуск и возврат

Команда без аргументов:

```
run
```

запускает текущую программу МК-61 так же, как последовательность F, `/-/`, В/О, С/П. В `.m61` это асинхронная точка ожидания: сценарий не идет дальше, пока программа МК-61 не остановится. Если программа ждет нажатий с клавиатуры, сценарий тоже ждет ее остановки.

После остановки программы выполнение продолжается со следующей строки `.m61`:

```
hin 0000 010203...
poke X 1.25e02
run
open RESULT.txt
```

Время каждого запуска программы измеряется автоматически. После останова последний результат в миллисекундах доступен через Система → Информация; дополнительная команда M61 для этого не требуется и экран калькулятора после останова не прерывается модальным сообщением.

Команда `ret` зависит от текущего контекста:

- в обработчике `trap` она восстанавливает калькулятор и продолжает программу с перехваченного шага;
- в обработчике `bind` она возвращает M61 к ожиданию следующего нажатия;
- в локальной метке, вызванной через `run :label` из обработчика `trap` или `bind`, она возвращает на строку после вызова;
- во вложенном `.m61` она немедленно возвращает в вызывающий сценарий;
- в корневом `.m61` без активных ловушек и привязок она завершает сценарий; при активных `trap` или `bind` оставляет наблюдатель включённым.

Достижение конца обычного корневого или вложенного файла сохраняет прежнее поведение завершения/возврата. Обработчики `trap` и `bind` обязаны выполнить явный `ret`: выход из них через конец файла считается ошибкой.

Ловушки программы МК-61

Полное техническое описание — от разбора строки до точки упреждающей выборки в ИК1302, снимка контекста и одноразового пропуска после `ret` — вынесено в отдельный документ «Команда `trap` в M61: семантика и реализация» (`MK61s-mini-M61-Trap.md`).

Ловушка объявляется строкой:

```
trap <address> run :label
```

Например:

```
trap 105 run :message

run
ret

:message
print "AT 105: X={X}\r\n"
ret
```

Строка `trap` активирует ловушку при своём выполнении. Это позволяет сначала запустить подготовительные программы, а затем объявить перехват только для основной программы. Все объявления и целевые метки всё равно проверяются при открытии файла: адрес не может повторяться, а отсутствующая метка является ошибкой до выполнения первой команды.

При повторном запуске того же корневого M61-файла уже активированные ловушки сохраняются. Поэтому команда `run` перед объявлениями `trap` выполняется без них только при первом запуске; повторный запуск использует прежнюю активную карту. Загрузка другого M61-файла, `ESC` или ошибка по-прежнему очищают её.

Адрес — десятичный номер opcode от `0` до `111`. В классическом режиме доступны шаги `0`...`104`; адреса `105`...`111` требуют расширенной памяти. Если активная ловушка лежит вне текущей программной памяти, следующий `run` завершается ошибкой. Для двухбайтовой команды указывается адрес её opcode, а не адрес байта-аргумента.

При достижении адреса ядро заканчивает предыдущую команду, но ещё не исполняет перехваченную. Полный контекст калькулятора сохраняется в оперативной памяти: кольцо, стек, регистры, X2, IP, режимы и внутреннее состояние микросхем. Затем сценарий переходит на метку. `ret` восстанавливает контекст байт-в-байт и разрешает ровно одно выполнение той же команды; при следующем приходе на адрес ловушка снова сработает. При ошибке или отмене сценария сохранённый контекст также восстанавливается.

Пока контекст сохранён, обработчику доступны только безопасные операции, которые не двигают и не изменяют калькулятор: `print`, `wait`, `ret`, `if`, `run :label`, `ver`, `list`, `dump`, `pub`, `lasm`, `isa`, `hout`, `reg`, `stk`, `1302`, `ring`, `led` и `beep`. Внутри обработчика `run :label` вызывает локальную метку и возвращается по её `ret`. Обычный `run`, `open`, `load`, `kbd`, `cmd`, `hin`, load <адрес> <путь.bin>, `asm`, `poke` и запись регистров в обработчике запрещены. Тот же фильтр временно применяется к параллельному интерактивному терминалу, чтобы команда с компьютера не изменила ядро между сохранением и `ret`.

Несколько адресов могут вести на одну метку:

```
trap 31 run :wall
trap 44 run :wall
```

Привязка клавиш `bind`

Формы привязок:

```
bind <opcode> run :label
bind key <код> down run :label
bind key <код> up run :label
bind timer run :label
```

`bind AE run :label` сохраняет прежний перехват введённой операции `AE`. Аргумент — две hex-цифры `00`…`EF`, как у `cmd`. Исходная операция заменяется на NOP; одинаковый opcode внутри программы не перехватывается. Допускается восемь таких привязок. Они работают в режиме ожидания клавиатурной операции.

`bind key` перехватывает физическую кнопку до её штатного действия, включая режим RUN. Код — ровно две hex-цифры из переносимой таблицы ниже, одинаковой для Mini, Classic и 40TH. Это логический код, не scan-code команды `kbd`. `down` вызывается один раз при нажатии, `up` — при отпускании; оба слова регистрозависимы и обязательны. Удержание не создаёт повторные `down`.

Код	Кнопка
`00`…`09`	Цифры 0…9
`0A`	Десятичная точка
`0B` / `0C` / `0D` / `0E`	+ / − / × / ÷
`0F` / `10`	Стрелки влево / вправо
`11` / `12`	Шаговые стрелки влево / вправо
`13`	ОК
`14`	ESC
`15`	С/П
`16`	Cx
`17` / `18`	К / F
`19`	USER
`1A` / `1B`	ПП / БП
`1C` / `1D`	x→П / П→x
`1E`	В/О
`21`	В↑
`22`	x↔y
`23`	ВП
`24`	/−/
`25` / `26` / `27`	Г / ГРД / Р
`28` / `29`	SAVE / LOAD

`1F` и `20` зарезервированы: в APP это дополнительные имена уже имеющихся шаговых кнопок. Для физических привязок используются `11` и `12`.

Каждой из 40 кнопок можно назначить два обработчика. Пара «код, событие» должна быть уникальна, а целевая метка обязана существовать. Лимит в восемь перехватчиков opcode не расходуется физическими привязками. Общие ограничения файла и числа меток остаются прежними. Все объявления проверяются при открытии сценария, а включаются только после исполнения соответствующей строки.

Например, R9 хранит удержание кнопки 4, а RA — запрос на один шаг влево:

```
bind key 04 down run :left_on
bind key 04 up run :left_off
load 0000 game.bin
run
ret

:left_on
R9= 1
RA= 4
ret

:left_off
R9= 0
ret
```

Игра выполняет запрос RA и сбрасывает его, а R9 использует для продолжения движения при удержании. Так короткое нажатие сохраняется, даже если кнопка успела отпуститься до следующего опроса регистров игровым циклом.

Перехватывается вся пара нажатие/отпускание, даже если назначен только один обработчик. Связанные кнопки не выполняют штатную операцию, не останавливают RUN и не меняют градусную меру. Несвязанные кнопки сохраняют прежнее действие.

Запрос во время RUN ждёт ближайшей границы команды МК-61. На время обработчика ядро приостановлено; `ret` продолжает выполнение без нового С/П. Изменённые обработчиком регистры и память не откатываются. Обработчик может сам вызвать `run`: тогда его вычисление продолжается до останова обычным образом.

Обработчики не вкладываются друг в друга. Нажатия и отпускания сохраняются в очереди, в том числе во время `wait`; во время `trap` они ждут восстановления снимка. Очередь резервирует место для отпускания принятого нажатия. Если она заполнена, новая пара целиком пропускается, а ранее принятый `up` не теряется. Для игрового цикла обработчики ввода следует делать короткими.

Привязки действуют на основном экране калькулятора вне ПРГ. Открытый просмотрщик или интерпретатор получает свою клавиатуру; старые удержания заканчиваются отпусканиями. После возврата из вложенного M61 старые удержания также завершаются.

Прежний `bind 0K run :help` больше не принимается. Его замена:

```
bind key 13 down run :help
```

`open? manual.md` по-прежнему ищет памятку рядом со сценарием. ESC внутри просмотрщика закрывает памятку. После корневого `ret` активные привязки продолжают работать; обычный останов калькулятора их не снимает. Другой корневой M61, ошибка, `reinit` или несвязанный ESC снимают привязки. При повторном открытии того же сценария сохраняются ещё существующие объявления.

`bind timer run :label` назначает обработчик таймера текущего сценария. Отсчёт запускает timer <миллисекунды>. Каждому запуску соответствует один вызов; для повторения обработчик может снова выполнить `timer`. `timer 0` отменяет ещё ожидающее срабатывание. Одно объявление таймера на файл проверяется вместе с остальными привязками и не занимает слот opcode.

```
bind timer run :expired
timer 4000
ret
:expired
print "TIME UP"
ret
```

Срок проверяет исполнитель M61: в `wait`, после корневого `ret` и между порциями обычных строк. Работающее ПЗУ приостанавливается на ближайшей границе команды, затем `ret` продолжает его. Обработчик таймера получает те же права записи, что и обработчик кнопки; регистры, используемые прерванной командной последовательностью, следует менять согласованно с программой. Таймер, клавиши и opcode не прерывают уже исполняющийся обработчик; `trap` и открытый просмотрщик/интерпретатор откладывают событие до возврата. `timer pause`/`resume` управляют тем же отсчётом, а вложенный M61 имеет свою привязку и свой таймер. Обработчик должен закончиться `ret`.

Долгое Cx остаётся системным выходом даже при связанном коротком Cx. На Mini/Classic/40TH оно завершает сценарий с физическими привязками, останавливает его калькулятор и возвращает обычный экран. Для сценариев только с перехватчиками opcode/trap/таймера прежнее снятие обработчиков

сохраняется. Снимаются все виды `bind` и `trap`, отменяется ожидающее срабатывание таймера, очищаются сохранённые кадры вложенных сценариев. Возврат из обработчика или вложенного файла не восстанавливает снятые привязки. Строка Cx сканируется и во время приостановки ядра обработчиком.

Повторная инициализация `reinit`

Команда

```
reinit
```

очищает программную память, стек, регистры памяти, префиксы, ошибку, адрес исполнения и расширенное состояние МК-61. Текущий внешний режим углов Р/Г/ГРД сохраняется. Также снимаются все активные `trap` и `bind`.

Внутри M61 текущий файл не перезапускается: выполнение продолжается со строки после `reinit`. Поэтому команду удобно ставить перед `hin` и начальными значениями регистров:

```
reinit
hin 0000 0102030450
R0= 1
run
```

В обработчике `trap` команда запрещена, потому что там калькулятор удерживается в сохранённом состоянии. В интерактивном терминале `reinit` сначала отменяет активный M61-сценарий, затем инициализирует калькулятор. Это отличается от `clr`, очищающего только программу, и от `rst`, перезагружающего весь микроконтроллер.

Временный шрифт `loadfont`

На графическом UC1609 команда `loadfont` временно выбирает внешний FMK для последующего вывода текста:

```
loadfont HighNoon
loadfont "High Noon" Pixel-12 DejaVu-12
```

Можно перечислить несколько имён через пробел. Они проверяются слева направо, и включается первый пригодный файл. Кавычки нужны только для имени с пробелами. Расширение `.FMK` необязательно.

Для каждого имени M61 сначала ищет файл шрифта в каталоге исполняемого сценария, затем в общем каталоге `Fonts`. Поэтому игра может хранить собственный шрифт рядом с `autoexec.m61`, а общий интерфейсный шрифт остаётся доступен из `Fonts`:

```
loadfont HighNoon Pixel-12
open intro.tbi
```

Вложенные команды `open` и вложенные `.m61` используют тот же сеанс. TinyBASIC, запущенный через `open`, наследует выбранный шрифт и соответствующие значения `COLS` и `ROWS`; в самом BASIC отдельной команды выбора шрифта нет.

`loadfont` без аргументов немедленно возвращает шрифт, который был активен до первой команды `loadfont` текущего корневого сценария:

```
loadfont HighNoon
open intro.tbi
loadfont
```

Обычный конец сценария, корневой `ret`, ошибка, отмена по ESC или запуск другого корневого `.m61` также всегда восстанавливают исходный шрифт. Поэтому сценарию не требуется отдельная команда очистки в конце.

Если ни один файл не найден, сценарий останавливается с ошибкой `font not found`. Повреждённый или несовместимый найденный FMK даёт `font file is invalid`; дисплей без поддержки этой функции — `loadfont requires UC1609`.

Вывод `print`

Две некавыченные формы управляют экраном из M61-сценария:

```
print off
print on
```

`print off` сразу очищает весь экран и передаёт его во владение текущего сценария: МК-61 продолжает считать, но не может писать на дисплей. `print on` снимает захват; прошивка сразу публикует текущую индикацию калькулятора. Обе формы работают только в `.m61`; строки `print "off"` и `print "on"` печатают обычный текст.

`print` принимает ровно одну строку в двойных кавычках и пишет её в текущий экранный буфер калькулятора. Команда ничего не добавляет к строке автоматически:

```
print "TEST"
print "X={X}, R0={R0}, X2={X2}\r\n"
print "\x1B[3;5H*"
print "\x1B[2K\rnew line"
print "14=\x14, 1B=\m1B"
print "\xFF"
```

Без суффикса подстановки имеют фиксированный индикаторный формат калькулятора:

Подстановка	Значение
`{X}`, `{Y}`, `{Z}`, `{T}`	стековые регистры
`{X1}`	скрытый стековый регистр X1
`{X2}`	отдельный регистр текущей индикации
`{R0}`...`{RE}`	регистры памяти
`{RF}`	RF в расширенном режиме
`{0}`...`{F}`	короткие синонимы `{R0}`...`{RF}`

Цифровые разряды подстановок проходят через ту же таблицу 16 состояний, что и индикатор МК-61:

Тетрада	Знак на экране
`0`...`9`	0, 1...9 (0 — экранный знак нуля)
A	-
B	L
C	C
D	Г
E	E
F	пробел

Таким образом, возникающие внутри регистра L, C, Г, E, минус и пустой разряд не превращаются в обычное десятичное число. `{X}`, `{Y}`, `{Z}`, `{T}`, `{X1}` и регистры памяти печатаются в фиксированном регистровом формате. Подстановка `{X2}` отличается от них: она копирует непосредственно текущие 12 разрядов индикатора вместе с положением запятой. Для вывода снятого кадра мультфильма МК-61 следует использовать именно её:

```
print "{X2}"
```

Для числового регистра или стека допустим короткий формат после двоеточия:

Формат	Что выводится
`{RA:m}`	первая цифра нормализованной мантиссы
`{RA:e}`	модуль десятичного порядка без ведущего нуля

Например, значение `RA = 3e-6` даёт 3 через `{RA:m}` и 6 через `{RA:e}`. Форматы регистронезависимы (`:M`, `:E` тоже допустимы); к индикаторной строке X2 они не применяются.

Имена подстановок регистронезависимы. {{ и }} печатают литеральные фигурные скобки. Поддерживаются escape-последовательности \\, \", \r, \n, \t, \b, \xHH и \mHH с ровно двумя hex-цифрами. Текстовые байты толкуются как M8: \x14 выводит круговую стрелку, \xFF — русскую «я». Для совместимости со старыми сценариями \x1B начинает ANSI-последовательность; буквальный знак M8 0x1B (надстрочное «ʸ») печатается через \m1B или непосредственно этим байтом в исходном файле. Неверный для M8 \x00 отображается символом замены.

ANSI разбирает сама прошивка до записи в активный дисплей — физический или USB Screen. Поддерживаются перемещения A/B/C/D/E/F, горизонтальная позиция G, позиционирование H/f, очистка J/K, сохранение и восстановление курсора s/u, а также ESC 7/ESC 8. Координаты ограничиваются фактическими cols() и rows() дисплея. SGR ...m принимается, но цвет и другие атрибуты на монохромном экране не имеют эффекта.

Обычный текст заменяет только занятые им ячейки. print не очищает ни строку, ни экран; очистка происходит только по явно напечатанным J или K.

После первого успешного print выполняющийся M61-сценарий временно владеет экраном: вычислительное ядро продолжает работать, но его промежуточные обновления индикатора не затирают сценарный GUI. Следующие print могут публиковать только выбранные кадры из обработчиков trap. При завершении или отмене корневого сценария обычная индикация калькулятора автоматически возвращается. Интерактивный print вне сценария экран не захватывает.

wait <миллисекунды> приостанавливает M61-сценарий на 1–60000 мс, не блокируя основной цикл прошивки. В обработчике trap снимок калькулятора остаётся приостановленным до окончания ожидания и последующего ret. Например, кадр можно удержать полсекунды:

```
:frame
print "{X2}"
wait 500
ret
```

Если калькулятор остановлен, обработчики bind key исполняются и во время обычного wait. После ret сценарий продолжает ждать до исходного срока: время обработчика входит в паузу. Собственный wait обработчика не меняет срок родительского ожидания. Другие привязки ждут завершения текущего обработчика; внутри trap клавиши по-прежнему откладываются до возврата сохранённого контекста.

Таймер по часам

timer <0..60000> запускает один таймер текущего сценария, не задерживая следующие команды. Значение задаётся в миллисекундах; timer 0 отменяет отсчёт. В условиях if источник timer возвращает оставшиеся миллисекунды, либо 0 после истечения срока или отмены:

```
timer 4000
:loop
wait 30
if timer>0 run :loop
print "TIME UP"
ret
```

Таймер использует монотонные часы прошивки и учитывает время расчётов ПЗУ и wait, включая переход счётчика через 32-битный ноль. Для ручного опроса условие проверяется при исполнении if. С привязкой bind timer run :label исполнитель сам вызывает обработчик по истечении срока. Видимый результат может дополнительно ждать отрисовки кадра.

timer pause сохраняет остаток, timer resume продолжает его. Повторная пауза или продолжение не меняют срок. Например, для памятки:

```
timer pause
open? manual.md
timer resume
```

Локальные метки, bind и trap используют таймер своего сценария. Вложенный файл M61 получает собственный, изначально отменённый таймер; при возврате восстанавливается таймер родителя, учитывающий прошедшее время. reinit, завершение и отмена корневого сценария очищают его таймер. Команда и источник timer добавлены 29.10.2026; нужна прошивка с этим расширением.

Метки и переходы

Метка - строка, которая начинается с двоеточия:

```
:loop
```

Метка сама ничего не выполняет. Переход на метку выполняется командой:

```
run :loop
```

При открытии файла метки один раз проверяются и индексируются, поэтому переход не перечитывает весь файл. Переход может быть и вперед, и назад; метка ищется только в текущем файле, не во вложенном сценарии. Имя метки содержит от 1 до 31 непробельного ASCII-символа, имена должны быть уникальны, максимум — 64 метки.

Пример цикла:

```
:again
run
if r0>0 run :again
beep 1200,80
```

Здесь программа МК-61 запускается снова, пока регистр `R0` больше нуля.

Условия

Команда `if` выполняет другую команду, только если условие истинно:

```
if <left><op><right> <command>
```

Поддерживаемые операции:

Операция	Значение
`>`	больше
`>=`	больше или равно
`<`	меньше
`<=`	меньше или равно
`==`	равно
`!=`	не равно

Операндами могут быть числа, регистры стека и регистры памяти:

Операнд	Что читает
`x`	стековый регистр X
`y`	стековый регистр Y
`z`	стековый регистр Z
`t`	стековый регистр T
`x1`	скрытый регистр X1
`r0`...`re`	регистры памяти R0...RE
`rf`	регистр RF, если включена расширенная память
`1.25e02`, `-5`, `0`	числовой литерал

После условия можно поставить любую обычную команду сценария, включая переход, запуск или открытие файла:

```
if x<0 open NEGATIVE.txt
if r3==10 run :done
if y!=0 run
if rf>=1.5e03 beep 2000,100
:done
```

Если условие ложно, строка ничего не делает и сценарий переходит к следующей строке.

Вложенные файлы

`open <path>` открывает сохраненный файл относительно папки текущего сценария:

```
open HELP.txt
open SETUP.m61
open TEST.foc
open SAMPLE.tbi
open ../lib/COMMON.m61
```

Для необязательного дополнения используйте `open? <path>`. Например, игра может показать памятку на сборке с Markdown и сразу перейти к загрузке на компактной сборке без `MARKDOWN.APP`:

```
open? manual.md
open part00.m61
```

Обычный `open` остаётся строгим и предназначен для обязательных частей: отсутствие `part00.m61` остановит сценарий с диагностикой. `open?` скрывает только отказ самого открытия; ошибка уже запущенного вложенного сценария не маскируется.

В `.m61` это работает как вызов с возвратом. Просмотрщик текста, FOCAL, TinyBASIC, шрифт или вложенный `.m61` выполняются, а после выхода управление возвращается к следующей строке исходного сценария.

Вложенный `.m61` сохраняет программные банки и регистры вызывающего сценария. Его `hin` меняют только указанные адреса, поэтому несколько файлов могут последовательно загрузить части одной программы. Для очистки используйте явный `reinit`. Открытие нового корневого M61 из Проводника или терминала по-прежнему очищает программную память перед выполнением файла.

`run <path>` - синоним `open <path>`:

```
run SETUP.m61
```

Для вложенных `.m61` есть стек возвратов на 8 уровней. Если глубина вложенности превышена или файл не найден, сценарий останавливается с ошибкой.

Файл с каноническим именем `autoexec.m61`, расположенный непосредственно в каталоге, служит точкой запуска этого каталога в Проводнике. При входе в папку коротким `OK` Проводник запускает сценарий тем же M61-runner и закрывается. Возврат `ESC`, файловые диалоги и начальное открытие корня автозапуск не вызывают; повторный вход в каталог запускает файл снова. Регистр имени значения не имеет.

Папка без `autoexec.m61` может тем же способом запускать `autoexec.tbi`. Если присутствуют оба файла, M61-сценарий имеет приоритет.

Команда:

```
load <number>
```

в интерактивном терминале загружает программу из корневого числового файла `M1`. В `.m61` она выполняет сохраненный сценарий `M1` с внутренним именем этого номера и возвращается назад. Например, `load 03` вызывает сценарий с именем 3. Для именованных файлов понятнее использовать `open <path>.m61`.

Работа со стеком и регистрами

Для записи в стек используется `poke`:

```
poke X 1.25e02
poke Y -3.14e00
poke Z 0e00
poke T 5e01
```

Имя стекового регистра - X, Y, Z или T. Значение задается обычным десятичным числом, например 3.14, -5 или 1.25e02. Допустимый порядок результата — от -99 до 99.

Регистры памяти можно читать в условиях через r0...re и rf. Запись в регистр памяти выполняется специальной командой:

```
R0= <value>
RA= <value>
RE= <value>
RF= <value>
```

RF доступен только в расширенном режиме памяти. R<r>= принимает такое же проверяемое десятичное число, как poke; напрямую испортить внутренние BCD-тетрады регистра числовой формой нельзя.

Для числового копирования другого регистра памяти укажите его имя справа:

```
R0= RA
R2= r0
```

Имена источников R0…RF регистронезависимы; RF доступен только в расширенном режиме. Значение проходит обычную проверку числа и записывается без изменения X и стека. Для переноса произвольных внутренних BCD-тетрад эта форма не предназначена. Копирование добавлено 29.10.2026 и доступно также в терминале.

Для независимой аппаратно инициализированной случайной дроби используется:

```
R0= random
```

В регистр записывается ненулевое число 0.0000001…0.9999999. Команда берёт значение из отдельного потока пула энтропии и работает одинаково при настройках К СЧ МК61 и К СЧ МК61s; состояние штатной команды К СЧ она не меняет. Источник рассчитан на игры и моделирование, а не на криптографию.

Исторические программы иногда хранят в регистре не число, а тетрады A…F. Для их точного и воспроизводимого запуска есть низкоуровневая форма:

```
RB= raw 8CEC6ABA9907
```

После raw должны идти ровно 12 шестнадцатеричных тетрад: восемь тетрад мантиссы, тетрада её знака, тетрада знака порядка и две хранимые тетрады порядка. Проверка всей строки выполняется до записи; соседние дорожки кольцевой памяти не затрагиваются.

Чтение таблиц lookup

```
lookup R5 200 R4
lookup R6 200 R4 R7 37 u24
@R5 200 R4
```

Команда М61 читает неизменяемые данные из расширенной памяти программы непосредственно в R0…RF. Короткая форма @ равнозначна lookup; пробел после @ необязателен. Обычная запись: lookup Rdst base index [row stride] [u24].

Без u24 читается один беззнаковый байт. С u24 читаются три байта, младший первым; значение составляет 0…16777215. Адрес равен base + width * (index + row * stride), где width — 1 или 3. Если row/stride опущены, их произведение равно нулю. База, индекс, строка и шаг допускают числовую константу либо операнд if (регистр, стек или timer), но обязаны быть целыми от 0 до 10000. Всё чтение должно помещаться в адреса 0000…9999; оно может пересекать банки. Невыделенная память читается как нули.

Меняется только регистр назначения: стек, остальные регистры и экран сохраняются. Источники читаются до записи, поэтому @R5 200 R5 разрешено. Неверный операнд или граница отвергается до изменения состояния. Нужен расширенный режим; команда запрещена в trap и при блокировке после ошибки загрузки BIN. Она доступна в М61, включая if и обработчики bind; интерактивный терминал её не выполняет.

Клавиатура и коды команд

kbd имитирует нажатие клавиши по scan-code:

```
kbd 02
kbd 27
```

Код задается в шестнадцатеричном виде и должен быть меньше 28h. В сценарии клавиша нажимается сразу в ядре эмулятора.

cmd имитирует ввод операции МК-61 по ее opcode:

```
cmd 10
cmd 14
```

Эта команда полезна, когда нужно выполнить ту же последовательность клавиш, которая соответствует байту программы МК-61. Для загрузки программы в память обычно лучше использовать hin или asm, а не cmd. Допустимый диапазон opcode для cmd — 00...EF.

Ассемблерный ввод

asm записывает программу мнемониками:

```
asm 0000 1 2 add hlt
```

После asm можно указать начальный адрес трансляции из четырех цифр. Если адрес не указан, используется текущий адрес трансляции. Мнемоники можно вывести командой isa.

Английские мнемоники можно смешивать с калькуляторными обозначениями в СР1251 и двухзначными hex-байтами. Например, Kx!=0 0 записывает один байт 70. Пробел между условием и регистром не создаёт вторую команду.

```
asm 0000 far jmp 02 50 disp 53 xmsr xmsp stoF ldF
```

Здесь far - префикс 1F, disp - префикс 2F, xmsr/xmsp - обмен Ms с регистрами/программой (55/56), а stoF/ldF - доступ к RF (4F/6F). Ввод far jmp 02 50 даёт 1F 51 02 50, то есть переход на абсолютный BCD-адрес 0250. Для disp следующий байт - hex-подкод индикатора, а не самостоятельная команда. Подробности приведены в руководстве терминала и описании расширений.

ins вставляет opcode в программу и сдвигает последующие команды:

```
ins 10 20
```

Первый аргумент - десятичный шаг программы, второй - hex-код команды.

set$ - служебная форма записи hex-строки:

```
set$0000 01020304
```

Для ручных .m61 предпочтительнее hin, потому что синтаксис читается однозначнее.

Справочник команд

Сценарные команды

Команда	Назначение
`run`	Запустить текущую программу МК-61 и ждать остановки.
`run :label`	Перейти на метку; внутри `trap`/`bind` — вызвать её с возвратом по `ret`.
`run <path>`	Открыть сохраненный файл, как `open <path>`.
`open <path>`	Открыть или запустить сохраненный файл с возвратом в сценарий.
`open? <path>`	Необязательно открыть файл; при недоступности молча продолжить.
loadfont **<имя>**...	Временно включить первый найденный FMK рядом со сценарием или в `Fonts`.
`loadfont`	Вернуть шрифт, активный до первого `loadfont` этого сценария.
load **<номер>**	В `.m61` выполнить сценарий `M1` с именем номера.
`load <path.m61>`	В интерактивном терминале загрузить M61 по пути.
if **<условие>** **<команда>**	Выполнить команду при истинном условии.
`trap <address> run :label`	Перехватить шаг программы до выполнения.
`bind <opcode> run :label`	Привязать введённую операцию к обработчику M61.
bind key **<код>** down/up run :label	Перехватить кнопку до штатной обработки, включая RUN.
`bind timer run :label`	Вызвать обработчик один раз по истечении `timer`.
`ret`	Вернуться из обработчика/вложенного файла или завершить корневой файл.
`print "..."`	Вывести строку, escapes и значения регистров.

Программная память МК-61

Команда	Назначение
`hin <addr> <hex>`	Записать hex-байты в программную память.
`lookup Rdst base index [row stride] [u24]`	Прочитать unsigned байт/слово из расширенной памяти прямо в регистр; короткая форма `@`.
`load <addr> <path.bin>`	Распаковать бинарный образ по абсолютному адресу.
`hout`	Вывести программную память hex-блоками.
`asm [addr] <mnemonics>`	Собрать мнемоники в программную память.
`isa`	Показать список мнемоник ассемблера.
`ins <step> <opcode>`	Вставить opcode в программу со сдвигом.
`list`	Показать программную память в hex-таблице.
`dump`	Показать программную память последовательностью hex-байтов.
`pub`	Показать листинг в журнальном формате.
`lasm`	Показать дизассемблированный листинг.
`clr`	Очистить программную память после подтверждения.
`set$<addr> <hex>`	Служебная запись hex-байтов.

Стек, регистры и выполнение

Команда	Назначение
`poke <stack> <value>`	Записать число в стековый регистр X, Y, Z или T.
`R<r>= <value>`	Записать число, `random` или точный `raw`-образ в R0...RE/RF.
`reg`	Показать регистры памяти R0...RE.
`stk`	Показать X1, T, Z, Y, X и текущий адрес программы.
`1302`	Показать регистр R микросхемы К145ИК1302.
`ring`	Показать активное кольцо памяти M.
`kbd <scan-code>`	Нажать клавишу по scan-code.
`cmd <opcode>`	Нажать последовательность клавиш, соответствующую opcode.
`reinit`	Очистить состояние калькулятора и снять активные `trap`/`bind`.

Хранилище

Команда	Назначение
`pwd`	Показать текущий каталог хранилища.
`cd [path]`	Перейти в каталог; без аргумента перейти в корень.
`ls [path]` / `dir` / `fsls`	Показать детей каталога либо один объект.
`mkdir [-p] <path>`	Создать каталог; `-p` создаёт отсутствующих родителей.
`mv <source> <destination>`	Перенести или переименовать без перезаписи.
`rm <path>` / `del` / `fsrm`	Удалить файл.
`rm -r <path>`	Рекурсивно удалить каталог со всем содержимым.
`rmdir <path>`	Удалить только пустой каталог.
`df` / `fsstat`	Показать ёмкость C6, узлы, файлы, каталоги и extents.
save **<номер>**	Сохранить текущую программу как числовой файл `M1` после подтверждения.
`save <path.m61>`	Сохранить текущую программу как M61 по пути после подтверждения.
load **<номер>** / `load <path.m61>`	Загрузить числовой либо путевой M61-файл.
`format`	Переразметить всё хранилище C6 после подтверждения.
`fsclean`	Удалить пустые записи хранилища.

Сервисные команды

Команда	Назначение
`ver`	Показать версию прошивки.
`help`	Показать список команд терминала.
`history`	Показать историю интерактивного терминала.
`disa`	Включить или выключить дизассемблер на дисплее.
`led <pattern>`	Запустить LED-паттерн, например `led 1,200,0`.
`beep <pattern>`	Запустить звуковой паттерн, например `beep 4000,100,0,50`.
`rst`	Перезагрузить микроконтроллер после подтверждения на устройстве.
`dfu`	Перейти в режим DFU.
`vlog`	Показать trace virtual FAT, если прошивка собрана с трассировкой.

Команды с подтверждением

Некоторые команды опасны или интерактивны:

```
clr
save <number|path.m61>
format
rst
dfu
```

Эти команды, а также `pwd`, `cd`, `ls`, `mkdir`, `mv`, `rm`, `rmdir`, `df`, `fsclean`, в `.m61` запрещены. Сценарий не может перезагрузить устройство, войти в DFU или административно изменять каталог. Для загрузки программы не нужно предварительно делать `clr`: строки `hin` записывают нужные байты напрямую.

Разрешения сценария

В `.m61` действует явный список разрешенных команд:

- управление потоком: `run`, `open`, `load`, `if`, `ret`, `trap`, `bind`;
- временный экранный шрифт: `loadfont`;
- программная память: `hin`, load <адрес> <путь.bin>, `set$`, `asm`, `ins`, `list`, `dump`, `pub`, `lasm`, `isa`, `hout`;
- калькулятор: `poke`, `R<r>=`, `reg`, `stk`, `1302`, `ring`, `kbd`, `cmd`, `reinit`;
- безопасный вывод и сигналы: `ver`, `print`, `wait`, `led`, `beep`.

Любая новая команда терминала по умолчанию запрещена для сценария, пока ее не добавят в этот список в прошивке. Это предотвращает случайное расширение прав файлов `.m61` при развитии терминала.

Ошибки выполнения

Сценарий останавливается, если:

- команда неизвестна;
- строка слишком длинная;
- аргументы команды разобрать нельзя;
- `open` или вложенный `load` не нашли файл;
- `loadfont` не нашёл пригодный шрифт либо не может включить его на текущем дисплее;
- вложенность `.m61` превысила 8 уровней;
- метка некорректна, повторяется или превышен лимит в 64 метки;
- ловушка некорректна, повторяется, ссылается на неизвестную метку либо не помещается в активную программную память;
- `bind` некорректен, повторяет opcode или пару «кнопка, событие», ссылается на неизвестную метку либо превышен лимит в 8 перехватчиков opcode;
- обработчик ловушки дошёл до конца файла без `ret`;
- обработчик `bind` дошёл до конца файла без `ret`;
- команда, запрещенная или невозможная в сценарном режиме, вернула ошибку.

После ошибки выполнение оставшихся строк не продолжается. Терминал сообщает имя сценария, номер строки и причину; последние сведения об ошибке также сохраняются в раннере до следующего запуска или явной отмены.

Примеры

Загрузить и запустить программу

```
hin 0000 014400450100004901OC0B024A09044C0108070207030806
hin 0024 0E0103000000000600384101030708070303060E01000006
hin 0048 000000003842010306020703080 60E010000000000060038
run
```

Подготовить стек, запустить и показать текст

```
poke X 1.25e02
poke Y 3.14e02
run
open RESULT.txt
```

Повторять программу по значению регистра

```
:loop
run
if r0>0 run :loop
beep 1200,80
```

Выбрать следующий шаг по стеку

```
run
if x<0 open NEGATIVE.txt
if x==0 open ZERO.txt
if x>0 open POSITIVE.txt
```

Вызвать другой сценарий и вернуться

```
open SETUP.m61
run
open CLEANUP.m61
```